SENIOR ARCHITECT MASTERCLASS

Multimodal GenAI

6 Topics across 6 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

Vision-Language Models

GPT-4o, Claude Vision, Gemini, LLaVA

> Imagine a person who can read books but has never seen a picture. That was LLMs until recently. Vision-Language Models (VLMs) gave LLMs EYES. Now you can show them a photo, a chart, a screenshot, or a handwritten note, and they can describe it, answer questions about it, and extract data from it. Like hiring an assistant who can both read AND see.

1. Image Understanding + Chart Reading + Use Cases

📖 DEEP TEACH

```
WHAT ARE VISION-LANGUAGE MODELS:
Imagine you have a brilliant assistant who has been blind their entire life. They can discuss anything in text -- history, science, code. But show them a photo and they say 'I cannot see images.'

VLMs gave this assistant perfect vision overnight. Now you can:
- Show them a photo: 'What is in this image?'
- Show them a chart: 'What are the trends in this bar chart?'
- Show them a screenshot: 'Extract all the text from this UI mockup.'
- Show them a receipt: 'What is the total amount?'
- Show them a diagram: 'Explain this architecture diagram.'

4 MAJOR VLMS:

GPT-4o (OpenAI):
The most capable all-rounder. Handles photos, charts, documents, screenshots. Can reason about what it sees. Accepts images via base64 or URL.
Best for: general image understanding, chart analysis, document processing.

CLAUDE VISION (Anthropic):
Similar capability to GPT-4o. Especially good at reading dense documents and long screenshots. Accepts base64 images.
Best for: document analysis, detailed image description.

GEMINI PRO VISION (Google):
Native multimodal -- designed from the ground up to handle text, images, audio, and video together. Large context window for multiple images.
Best for: multi-image comparison, long document processing.

LLAVA (open-source):
Imagine a free alternative you can run on your own servers. Not as good as GPT-4o but FREE and private. Your images never leave your network.
Best for: privacy-sensitive image processing, on-premise deployment.

HOW THEY WORK (simple version):
The image is converted into a grid of visual 'tokens' (like words, but for pictures). These visual tokens are fed into the LLM alongside text tokens. The LLM processes both together.

A single image uses about 85-1000 tokens depending on size and detail. That means image analysis costs more than text-only queries.

USE CASES:
- OCR on steroids: read handwritten notes, receipts, whiteboards
- Chart data extraction: 'What was Q3 revenue according to this chart?'
- UI testing: screenshot comparison, accessibility audit
- Medical imaging: (with proper compliance) X-ray preliminary analysis
- Document processing: extract structured data from invoices, contracts
- Diagram understanding: explain architecture diagrams, flowcharts
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of VLMs like giving your assistant glasses for the first time. Suddenly they can do things that were impossible before -- but glasses cost extra (image tokens are expensive).

1. GPT-4o is the default for image understanding. Best quality-to-ease ratio.
2. IMAGE TOKENS: a single image costs 85-1000+ tokens. At scale, this adds up fast. Resize images to the minimum needed resolution.
```

3. DETAIL PARAMETER: OpenAI has 'detail: low' (85 tokens, cheap, for simple images) vs 'detail: high' (up to 1000+ tokens, for complex charts). Use 'low' when you can.
4. MULTI-IMAGE: send multiple images in one call for comparison. 'Which of these 3 designs is best?'
5. LLaVA for privacy: images processed locally. No data sent to cloud.
6. STRUCTURED EXTRACTION: combine vision with instructor/Pydantic to extract typed data from images.

WAR STORY: Insurance company manually processed 10,000 claim photos per week (damage assessments). Each took 5 minutes. Deployed GPT-4o vision: each photo processed in 3 seconds with 92% accuracy. Human reviewers now only check the 8% the model flags as uncertain. Processing time: 833 hours/week -> 40 hours/week.

📋 POCKET NOTES

VLMs: LLMs that can see. Image -> visual tokens -> processed with text.
GPT-4o: best all-rounder. Photos, charts, docs, screenshots.
Claude Vision: great for dense documents and long screenshots.
Gemini: native multimodal. Multi-image. Large context.
LLaVA: open-source, free, run locally. Privacy-friendly.
Image tokens: 85 (low detail) to 1000+ (high detail) per image.
Use 'detail: low' when possible to save cost.
Use cases: OCR, chart extraction, UI testing, document processing.
Combine with instructor for typed data extraction from images.
Resize images to minimum needed resolution (save tokens).

🎯 INTERVIEW PREP

SAY THIS: 'I use GPT-4o for general image understanding and Claude Vision for dense document analysis. For privacy-sensitive use cases, LLaVA runs locally. The key cost consideration is image tokens -- I use low-detail mode when possible and resize images to minimize token usage.'

Q: How do you extract structured data from images?
A: 'Vision model describes the image, then I pipe the description through instructor with a Pydantic schema for typed extraction. For charts, I ask the model to extract the data points as JSON. For invoices, I define an Invoice schema and the model fills it from the image.'

Q: GPT-4o vs Claude Vision vs Gemini for images?
A: 'GPT-4o: best general quality. Claude: best for long documents with many details. Gemini: best for multi-image comparison and video frames. LLaVA: best for privacy (runs locally). I benchmark on my specific image types before choosing.'

⌨️ CODE

Python

```python
from openai import OpenAI
import base64
client = OpenAI()

# Method 1: Image from URL
response = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role":"user","content":[
        {"type":"text","text":"What is in this image? Describe in detail."},
        {"type":"image_url","image_url":{
            "url":"https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg",
            "detail":"low"  # 85 tokens. Use 'high' for complex images.
        }}
    ]}]
)
print(f"Image description: {response.choices[0].message.content[:100]}...")

# Method 2: Image from file (base64)
# with open('chart.png', 'rb') as f:
#     img_b64 = base64.b64encode(f.read()).decode()
# Then use: {"type":"image_url","image_url":{"url":f"{img_b64}"}}

# Method 3: Extract structured data from image
print("\
Structured extraction:")
response2 = client.chat.completions.create(
    model="gpt-4o",
```

```
    response_format={"type":"json_object"},
    messages=[
        {"role":"system","content":"Extract data as JSON."},
        {"role":"user","content":[
            {"type":"text","text":"Extract: subject, colors, setting from this image. JSON only."},
            {"type":"image_url","image_url":{
                "url":"https://upload.wikimedia.org/wikipedia/commons/thumb/3/3a/Cat03.jpg/1200px-Cat03.jpg"}}
        ]}
    ]
)
print(response2.choices[0].message.content)
```

📺 **OUTPUT**

```
Image description: The image shows a tabby cat with green eyes sitting on a wooden surface...

Structured extraction:
{"subject": "tabby cat", "colors": ["brown", "black", "white"], "setting": "wooden surface, indoor"}

# The LLM can SEE the image and extract structured data.
# detail:'low' = 85 tokens (cheap). detail:'high' = 1000+ tokens (detailed).
```

Image Generation

Stable Diffusion, DALL-E 3, diffusers, ControlNet

Imagine describing a painting to an artist and they create it from your words alone. 'A cozy cabin in the mountains at sunset with snow on the roof.' The artist paints exactly that. Image generation models are that artist -- they turn text descriptions into images. And with tools like ControlNet, you can guide the artist even more precisely.

2. Text-to-Image + ControlNet + img2img + Inpainting

📖 DEEP TEACH

```
HOW TEXT-TO-IMAGE WORKS (the simple version):
Imagine a TV showing pure static (random noise). Now imagine SLOWLY tuning the TV until a clear picture appears. That is literally
how diffusion models work.

Step 1: Start with random noise (static)
Step 2: The model gradually REMOVES noise, guided by your text description
Step 3: After many steps of 'denoising,' a clear image appears

The text description ('a cat on a beach') steers what the denoised image looks like.

3 MAJOR MODELS:

STABLE DIFFUSION (open-source king):
Free, runs on your GPU, fully customizable. Huge community.
SD 1.5, SDXL, SD3. The diffusers library from HuggingFace makes it easy.
Best for: full control, custom training, privacy, no usage limits.

DALL-E 3 (OpenAI, via API):
Best at following complex text prompts accurately. Understands nuance.
No local setup needed. Pay per image (~$0.04-0.08 each).
Best for: quick, high-quality generation without managing GPUs.

MIDJOURNEY (aesthetic champion):
Produces the most visually stunning images. Discord-based interface.
Not available as a traditional API. Design-focused.
Best for: marketing visuals, artistic content.

4 TECHNIQUES:

TEXT-TO-IMAGE (basic): describe -> get image
  'A golden retriever wearing sunglasses on a beach'

IMAGE-TO-IMAGE (img2img): provide a starting image + description -> modified image
  Like giving the artist a rough sketch and saying 'make it look like this but in watercolor style.'

INPAINTING: mark a region of an existing image -> replace just that region
  Like cutting out someone's hat in a photo and asking the artist to paint a crown instead.

CONTROLNET: provide a structural guide (edges, pose, depth map) + description
  Like giving the artist a wireframe skeleton and saying 'draw a person in this exact pose wearing a red dress.' The pose is
locked, the creative details are free.
  Essential for: consistent character poses, architectural layouts, product placement.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of image generation like commissioning art. DALL-E 3 is a professional studio (easy, pay per piece). Stable Diffusion is
setting up your own art studio (more work upfront, unlimited pieces). Midjourney is a famous artist's gallery (beautiful but their
process, not yours).

1. DALL-E 3: fastest path to production. API call, no GPU needed.
2. Stable Diffusion + diffusers: full control, free, but need a GPU (RTX 3090+ recommended).
3. PROMPT ENGINEERING for images is different from text. Be descriptive: 'cinematic lighting, 4K, detailed, professional
photography' helps a lot.
4. NEGATIVE PROMPTS: tell the model what NOT to draw. 'bad anatomy, blurry, low quality.' Massively improves results.
5. ControlNet: the game-changer for production. Consistent poses, layouts, and compositions.
```

6. SAFETY: all models have content filters. Stable Diffusion can be uncensored (responsibility is on you).
7. COST: DALL-E 3 ~$0.04/image. Stable Diffusion on your GPU: ~$0.001/image after hardware cost.

WAR STORY: E-commerce company needed product images on different backgrounds. Manual photography: $50/product, 3 days turnaround. Stable Diffusion inpainting: replace background in 5 seconds for ~$0.001/image. Quality was 90% as good. Used AI for catalog images and professional photos only for hero images.

📋 POCKET NOTES

How it works: start with noise, gradually denoise guided by text description.
Stable Diffusion: open-source, free, your GPU. Full control. diffusers library.
DALL-E 3: OpenAI API. Best prompt following. ~$0.04-0.08/image.
Midjourney: most aesthetic. Discord-based.
4 Techniques:
  Text-to-image: describe -> get image
  img2img: starting image + changes -> modified image
  Inpainting: mark region -> replace just that part
  ControlNet: structural guide (pose/edges) + description -> consistent output
Prompt tips: be descriptive, include style words, use negative prompts
Cost: DALL-E ~$0.04/img | SD on GPU ~$0.001/img
pip install diffusers transformers accelerate

🎯 INTERVIEW PREP

SAY THIS: 'For production image generation, I choose based on control needs. DALL-E 3 for quick, high-quality results via API. Stable Diffusion with ControlNet for consistent, controllable generation at scale. The key production technique is ControlNet -- it locks structure while allowing creative variation.'

Q: How do you ensure consistent image style across a product?
A: 'ControlNet with a fixed structural guide (edge detection or pose estimation) plus a consistent style prompt and seed. This gives the same composition and style while allowing content variation. For brand consistency, I fine-tune Stable Diffusion on brand images.'

Q: DALL-E 3 vs Stable Diffusion for production?
A: 'DALL-E 3: zero infrastructure, best prompt understanding, pay per image. Stable Diffusion: full control, unlimited generation, but need GPU infrastructure. Crossover: at ~1000 images/day, self-hosted SD becomes cheaper.'

⌨ CODE

Python

```python
# DALL-E 3: Generate image via API (simplest)
from openai import OpenAI
client = OpenAI()

response = client.images.generate(
    model="dall-e-3",
    prompt="A cozy mountain cabin at sunset with snow on the roof, warm light from windows, cinematic photography",
    size="1024x1024",
    quality="standard",  # or 'hd' for higher quality
    n=1
)
print(f"Image URL: {response.data[0].url[:80]}...")
print(f"Cost: ~$0.04 per image\
")

# Stable Diffusion: Generate locally (full control)
# pip install diffusers transformers accelerate torch
print("""# Stable Diffusion (local GPU):
from diffusers import StableDiffusionPipeline
import torch

pipe = StableDiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16
).to("cuda")

image = pipe(
    prompt="A cozy mountain cabin at sunset, cinematic, 4K",
```

```
    negative_prompt="blurry, low quality, bad composition",
    num_inference_steps=30,
    guidance_scale=7.5,
).images[0]

image.save("cabin.png")
# Cost: ~$0.001 per image (electricity only)
# Full control: custom models, ControlNet, inpainting""")

print("\
# ControlNet (structural control):")
print("# 1. Extract edges/pose from reference image")
print("# 2. Feed edges + text prompt to ControlNet")
print("# 3. Result: same structure, different content")
print("# Like tracing a coloring book outline but painting freely inside")
```

📺 **OUTPUT**

```
Image URL: https://oaidalleapiprod.blob.core.windows.net/private/...
Cost: ~$0.04 per image

# Stable Diffusion (local GPU): ~$0.001/image, full control
# ControlNet: lock structure (pose/edges), vary content
# Like giving an artist a wireframe to follow
```

Audio & Speech

Whisper (STT), TTS, voice cloning, real-time agents

> Imagine your LLM could HEAR and SPEAK. Whisper gives it ears (converts speech to text). Text-to-Speech gives it a voice (converts text to speech). Together, you can build voice agents that have phone conversations, transcribe meetings, or narrate content -- all powered by AI.

3. Speech-to-Text + Text-to-Speech + Voice Agents

📖 DEEP TEACH

```
3 AUDIO CAPABILITIES:

1. SPEECH-TO-TEXT (Whisper -- giving the AI ears):
Imagine a court stenographer who types every word spoken in real-time. But instead of a human, it is an AI.

OpenAI Whisper: the gold standard. Handles 100+ languages, accents, background noise, and even speaker identification. Available
as: API ($0.006/minute), or self-hosted (free, run locally with whisper library).

Use cases: meeting transcription, podcast indexing, voice input for chatbots, accessibility.

2. TEXT-TO-SPEECH (giving the AI a voice):
Imagine reading a bedtime story to a child, but instead of your voice, an AI reads it in any voice you choose -- a warm narrator, a
news anchor, or even a custom voice.

OPENAI TTS: 6 built-in voices. Simple API. Good quality. $15/1M characters.
ELEVENLABS: premium quality. Voice cloning from 30 seconds of audio. Most natural-sounding. $5-330/month.
COQUI TTS: open-source. Run locally. Free. Less natural but improving fast.

3. REAL-TIME VOICE AGENTS (the full conversation):
Imagine calling customer service and talking to an AI that sounds human, understands context, and resolves your issue -- all in
real-time.

Pipeline: User speaks -> Whisper (speech-to-text) -> LLM (thinks) -> TTS (text-to-speech) -> User hears

The challenge is LATENCY. Each step takes time:
  Whisper: 500ms | LLM: 1-3s | TTS: 300ms | Total: 2-4 seconds

For real-time conversation, you need total latency under 1 second. Solutions:
  - Stream everything (start TTS before LLM finishes)
  - Use faster models (Groq for LLM, smaller Whisper model)
  - Optimize chunk sizes

OpenAI Realtime API: handles the entire voice pipeline with optimized latency.
LiveKit + OpenAI: production framework for building voice agents.

VOICE CLONING (ethical considerations):
ElevenLabs can clone a voice from 30 seconds of audio. Powerful for:
  - Custom brand voices
  - Audiobook narration in the author's voice
  - Accessibility (reading in a familiar voice)
But also risky: deepfakes, impersonation, fraud. Always get consent. Many jurisdictions have laws about this.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of voice AI like building a phone system. Whisper is the microphone (hears). LLM is the brain (thinks). TTS is the speaker
(talks). You need all three for a conversation.

1. Whisper API: $0.006/minute. Cheapest and easiest. Self-hosted: free but needs GPU.
2. ElevenLabs: best voice quality but expensive. Use for customer-facing voices.
3. OpenAI TTS: good balance of quality and cost. 6 voices, simple API.
4. LATENCY is the #1 challenge for real-time voice. Target: under 1 second total.
5. STREAMING: start speaking (TTS) before the LLM finishes thinking. Stream sentence by sentence.
6. Voice cloning: ALWAYS get consent. Legal and ethical requirement.

WAR STORY: Call center deployed a voice agent for tier-1 support. Initial latency: 3.5 seconds (users kept saying 'hello? are you
```

there?'). Optimized: Groq for fast LLM (200ms), streaming TTS, smaller Whisper model. Latency dropped to 800ms. User satisfaction matched human agents for simple queries.

📋 POCKET NOTES

```
Speech-to-Text: Whisper. 100+ languages. API $0.006/min. Self-host: free.
Text-to-Speech:
  OpenAI TTS: 6 voices, $15/1M chars. Good balance.
  ElevenLabs: best quality, voice cloning. $5-330/month.
  Coqui TTS: open-source, free, run locally.
Voice Agent pipeline: speak -> Whisper -> LLM -> TTS -> hear
Latency challenge: target < 1 second total
  Stream everything. Groq for fast LLM. Smaller Whisper model.
OpenAI Realtime API: optimized voice pipeline.
Voice cloning: 30 seconds of audio. ALWAYS get consent.
Ethical: deepfakes, impersonation risk. Legal requirements.
```

🎯 INTERVIEW PREP

SAY THIS: 'For voice agents, the pipeline is Whisper for speech-to-text, LLM for reasoning, and TTS for speech output. The critical engineering challenge is latency -- I target under 1 second by streaming each component and using fast inference like Groq.'

Q: How do you build a real-time voice agent?
A: 'Pipeline: Whisper -> LLM -> TTS with streaming at every step. Key optimizations: start TTS before LLM finishes (stream sentence by sentence), use Groq or a fast local model, and smaller Whisper model for real-time. OpenAI Realtime API handles this pipeline natively.'

Q: Voice cloning concerns?
A: 'Always get explicit consent. Many jurisdictions have laws. Use cases: brand voices (consented), audiobook narration, accessibility. Never for impersonation. Implement voice watermarking when possible.'

⌨️ CODE

Python

```python
# Speech-to-Text: Whisper API
from openai import OpenAI
client = OpenAI()

# Transcribe audio file
# audio_file = open("meeting.mp3", "rb")
# transcript = client.audio.transcriptions.create(
#     model="whisper-1",
#     file=audio_file
# )
# print(f"Transcript: {transcript.text}")
# Cost: $0.006 per minute

print("Whisper API: $0.006/min for 100+ languages\
")

# Text-to-Speech: OpenAI TTS
response = client.audio.speech.create(
    model="tts-1",       # or tts-1-hd for higher quality
    voice="nova",         # alloy, echo, fable, onyx, nova, shimmer
    input="Hello! I am an AI voice agent. How can I help you today?"
)
# response.stream_to_file("output.mp3")
print("TTS: 6 voices available. $15/1M characters.\
")

# Voice Agent Pipeline (conceptual)
print("REAL-TIME VOICE AGENT PIPELINE:")
print("  1. User speaks into microphone")
print("  2. Whisper converts speech -> text (500ms)")
print("  3. LLM processes query (1-3s, or 200ms with Groq)")
print("  4. TTS converts response -> speech (300ms)")
print("  5. User hears the response")
print("  Total: 2-4s naive | <1s optimized (streaming + Groq)")
```

```
print("\
Full pipeline frameworks:")
print("  - OpenAI Realtime API (native voice pipeline)")
print("  - LiveKit + OpenAI (production voice agents)")
print("  - Pipecat (open-source voice agent framework)")
```

📺 **OUTPUT**

```
Whisper API: $0.006/min for 100+ languages

TTS: 6 voices available. $15/1M characters.

REAL-TIME VOICE AGENT PIPELINE:
  1. User speaks into microphone
  2. Whisper converts speech -> text (500ms)
  3. LLM processes query (1-3s, or 200ms with Groq)
  4. TTS converts response -> speech (300ms)
  5. User hears the response
  Total: 2-4s naive | <1s optimized (streaming + Groq)

# Voice agents are becoming the new phone bots.
# Key: latency optimization through streaming everything.
```

Video Generation

Sora, Runway, Kling — emerging field

> Imagine image generation but with a TIME dimension. Instead of creating one picture, you create a moving sequence of pictures -- a video. This is the newest and most rapidly evolving area of generative AI. Think of it as giving the AI artist not just a canvas but a movie camera.

4. Current Models + Architecture + Practical Reality

📖 DEEP TEACH

```
THE STATE OF VIDEO GENERATION (honest assessment):

Imagine the early days of image generation -- impressive demos but not quite production-ready for most use cases. That is where
video generation is right now.

CURRENT MODELS:

SORA (OpenAI):
The most hyped. Can generate 60-second videos from text prompts. Impressive quality.
Limited availability. Very expensive to run. Not yet a production tool for most teams.

RUNWAY GEN-3:
Most accessible for production use. Text-to-video and image-to-video.
Web interface and API. Used in film and advertising.
Best for: creative professionals, marketing videos, prototyping.

KLING (Kuaishou, China):
Competitive quality. Longer videos. Growing fast.

STABLE VIDEO DIFFUSION (open-source):
Image-to-video (animate a still image). Shorter clips.
Run locally. Good for experimentation.

HOW IT WORKS (simplified):
Same idea as image generation but in 3D (width x height x TIME).
1. Start with noise (like image generation)
2. Denoise frame by frame, maintaining consistency across frames
3. The hardest part: temporal consistency (the chair should not change shape between frames)

PRACTICAL REALITY CHECK:
- Generation is SLOW (minutes to hours per video)
- Quality is inconsistent (physics violations, morphing artifacts)
- Very expensive (GPU-intensive)
- Short clips (5-60 seconds typically)
- Not yet reliable enough for most production use cases
- Best current use: creative exploration, prototyping, marketing concepts

WHERE IT IS HEADED:
- Real-time generation (not there yet)
- Longer, more coherent videos
- Better physics understanding
- Integration with editing tools
- Likely 2-3 years from mainstream production readiness
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of video generation like self-driving cars in 2015. Impressive demos. Real promise. But not ready for production in most
scenarios. Know what it can and cannot do.

1. BE HONEST about limitations. Video gen is impressive but NOT production-ready for most use cases.
2. Runway Gen-3 is the most practical option today for short creative clips.
3. Stable Video Diffusion for experimentation (free, local).
4. USE CASES that work TODAY: short social media clips, product animations, concept visualization, B-roll footage.
5. USE CASES NOT ready: long-form content, real-time generation, precise control.
6. KEEP WATCHING this space. It is evolving monthly.
```

WAR STORY: Marketing team tried using Sora for a product launch video. After 3 days of prompt tweaking, the 30-second video had the product morphing slightly between frames and a hand with 6 fingers. They used it as a social media teaser (the slight imperfection was 'charming') but filmed the main video traditionally. Honest assessment: great for ideation, not for final production. Yet.

📋 POCKET NOTES

Sora (OpenAI): most hyped. 60s videos. Limited access. Not production-ready.
Runway Gen-3: most accessible. Text/image to video. Creative professionals.
Kling: competitive quality. Longer clips.
Stable Video Diffusion: open-source. Image-to-video. Experimentation.
How it works: like image gen but with time dimension. Denoise frame by frame.
Reality check:
  Slow (minutes-hours per video)
  Inconsistent quality (physics violations)
  Expensive (GPU-intensive)
  Short clips (5-60 seconds)
Works today: social clips, product animations, concept viz
Not ready: long-form, real-time, precise control
2-3 years from mainstream production readiness.

🎯 INTERVIEW PREP

SAY THIS: 'Video generation is impressive but not production-ready for most use cases. Runway Gen-3 is the most practical for short creative clips. I am honest about limitations -- temporal consistency, physics violations, and generation time are real constraints. I track this space monthly because it evolves rapidly.'

Q: Should we use video generation in production?
A: 'For specific narrow use cases, yes: social media teasers, product animations, concept visualization, B-roll. For polished long-form content: not yet. The quality is not consistent enough. I recommend a hybrid approach -- AI for ideation and rough cuts, human refinement for final output.'

Q: When will video gen be production-ready?
A: 'For short creative clips: now (Runway). For reliable long-form content: likely 2-3 years. For real-time generation: 3-5 years. The trajectory is similar to image generation, which went from novelty to production-grade in about 3 years (2022-2025).'

⌨️ CODE

Python

```python
# Video generation is mostly API/cloud-based today
# Here is how you would use the key tools:

print("=== VIDEO GENERATION LANDSCAPE ===")
print()
print("1. DALL-E + Animation (simplest approach):")
print("   Generate key frames with DALL-E 3")
print("   Interpolate frames with Stable Video Diffusion")
print("   Result: short animated clips from text")
print()

# Runway API (most production-ready)
print("2. RUNWAY GEN-3 API:")
print("""   import requests
   response = requests.post(
       'https://api.runwayml.com/v1/generate',
       headers={'Authorization': 'Bearer YOUR_KEY'},
       json={
           'prompt': 'A golden retriever running on a beach at sunset',
           'duration': 4,  # seconds
           'resolution': '1280x768'
       }
   )
   # Returns: video URL after processing (1-5 minutes)
""")

# Stable Video Diffusion (local, free)
print("3. STABLE VIDEO DIFFUSION (local):")
print("""   # pip install diffusers transformers accelerate
```

```
    from diffusers import StableVideoDiffusionPipeline
    import torch

    pipe = StableVideoDiffusionPipeline.from_pretrained(
        'stabilityai/stable-video-diffusion-img2vid',
        torch_dtype=torch.float16
    ).to('cuda')

    # Animate a still image into a short video
    from PIL import Image
    image = Image.open('dog_on_beach.png').resize((1024, 576))
    frames = pipe(image, num_frames=25).frames[0]
    # Result: 25 frames (~1 second of video)
""")

print("COST COMPARISON:")
print("  Runway Gen-3: $0.05-0.50 per second of video")
print("  Stable Video Diffusion: free (your GPU electricity)")
print("  Sora: not publicly priced yet")
```

📺 **OUTPUT**

```
=== VIDEO GENERATION LANDSCAPE ===

1. DALL-E + Animation: generate frames, interpolate
2. Runway Gen-3: most production-ready API
3. Stable Video Diffusion: local, free, image-to-video

COST COMPARISON:
  Runway Gen-3: $0.05-0.50 per second of video
  Stable Video Diffusion: free (your GPU electricity)
  Sora: not publicly priced yet

# Honest take: great for short creative clips today.
# Not ready for long-form production content. Yet.
```

Multimodal Embeddings

CLIP, SigLIP — same space for images and text

Remember how text embeddings convert words into numbers where similar meanings have similar numbers? Now imagine doing the same but for BOTH text AND images in the SAME number space. The text 'a photo of a dog' and an actual photo of a dog get SIMILAR numbers. This lets you search images with text and text with images.

5. CLIP + SigLIP + Cross-Modal Search

📖 DEEP TEACH

```
THE MAGIC OF MULTIMODAL EMBEDDINGS:

Imagine a universal translator that converts BOTH English and Japanese into the SAME secret code. If someone writes 'dog' in English and another writes the Japanese word for dog, they get the same code number. Now you can match English to Japanese without knowing either language -- just compare the codes.

Multimodal embeddings do this for text and images:
  Text: 'a photo of a golden retriever' -> [0.82, -0.31, 0.55, ...]
  Image: (actual photo of a golden retriever) -> [0.80, -0.33, 0.54, ...]
  (Numbers are very SIMILAR!)

  Text: 'a red sports car' -> [0.12, 0.78, -0.45, ...]
  (Numbers are very DIFFERENT from the dog numbers)

THIS ENABLES:
- TEXT-TO-IMAGE SEARCH: 'Find me photos of sunset beaches' -> returns beach images
- IMAGE-TO-TEXT SEARCH: upload a photo -> find articles about what is in it
- IMAGE-TO-IMAGE SEARCH: upload a photo -> find similar photos
- ZERO-SHOT CLASSIFICATION: 'Is this image a cat, dog, or bird?' without any training

KEY MODELS:

CLIP (OpenAI, 2021):
The pioneer. Trained on 400 million image-text pairs from the internet. Two encoders: one for text, one for images. Both produce vectors in the same space.
Still widely used. Many variants and improvements built on top.

SigLIP (Google, 2023):
Improved version of CLIP. Better training objective (sigmoid loss instead of softmax). Better performance on retrieval tasks.

Jina-CLIP (Jina AI):
Optimized for production. Supports long text (8K tokens). Good for RAG with images.

OpenCLIP:
Open-source reproduction of CLIP with various model sizes. Community-trained on larger datasets.

HOW THEY ARE USED IN PRACTICE:
1. MULTIMODAL RAG: embed both text chunks AND images in the same vector database. Search finds both.
2. IMAGE SEARCH: build Google-like image search for your own data.
3. CONTENT MODERATION: check if an image matches prohibited categories.
4. E-COMMERCE: 'find products that look like this photo.'
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of CLIP like a bilingual person who speaks both 'image language' and 'text language' fluently. They can translate between the two.

1. CLIP is the foundation. Most multimodal AI builds on it.
2. For production search: Jina-CLIP or SigLIP (better than original CLIP).
3. ZERO-SHOT CLASSIFICATION is incredibly powerful: classify images into ANY categories without training. Just compare image embedding to text embeddings of category names.
4. Same vector database (Qdrant, Pinecone) can hold both text AND image embeddings.
5. Embedding dimension: typically 512-768 for CLIP models.
6. LIMITATION: CLIP understands WHAT is in an image but struggles with spatial relationships ('the cat is TO THE LEFT of the dog').
```

WAR STORY: E-commerce company wanted visual search: user uploads a photo of a dress they like, system finds similar dresses in the catalog. Before CLIP: custom CNN model, 6 months of training, 82% accuracy. With CLIP: 2 days of embedding their catalog, 89% accuracy. Zero training. Just embed images and text in the same space.

📋 POCKET NOTES

Multimodal embeddings: text AND images in the SAME number space.
  'photo of a dog' (text) and (actual dog photo) get similar numbers.
CLIP (OpenAI): pioneer. 400M image-text pairs. Two encoders.
SigLIP (Google): improved CLIP. Better retrieval.
Jina-CLIP: production-optimized. Long text (8K). Good for RAG.
Enables: text-to-image search, image-to-text, zero-shot classification.
Zero-shot: classify images into ANY categories without training.
Same vector DB holds both text + image embeddings.
Dimensions: 512-768 typically.
Limitation: CLIP understands WHAT, not spatial WHERE.

🎯 INTERVIEW PREP

SAY THIS: 'I use multimodal embeddings to put text and images in the same vector space. This enables cross-modal search, zero-shot image classification, and multimodal RAG where both text chunks and images are searchable from one query.'

Q: How does zero-shot image classification work?
A: 'Embed the image with CLIP. Embed the category names as text. Compare distances. The closest text embedding IS the classification. No training needed. Just provide category names.'

Q: How do you build visual search?
A: 'Embed all product images with CLIP. Store in a vector DB. When user uploads a photo, embed it with the same model and find nearest neighbors. Took 2 days vs 6 months for a custom model.'

⌨️ CODE

Python

```python
# CLIP: embed text and images in the same space
# pip install transformers torch pillow

from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch
import numpy as np

# Load CLIP
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

# Embed text descriptions
texts = ["a photo of a cat", "a photo of a dog", "a red sports car"]
text_inputs = processor(text=texts, return_tensors="pt", padding=True)
text_embeds = model.get_text_features(**text_inputs)
text_embeds = text_embeds / text_embeds.norm(dim=-1, keepdim=True)  # Normalize

print("Text embeddings shape:", text_embeds.shape)  # (3, 512)

# Embed an image (simulated - in production, load real image)
# image = Image.open("cat.jpg")
# image_inputs = processor(images=image, return_tensors="pt")
# image_embeds = model.get_image_features(**image_inputs)

# Zero-shot classification (no training needed!)
print("\
ZERO-SHOT CLASSIFICATION:")
print("Given an image, which text description matches best?")
print("Just compare the image embedding to each text embedding.")
print("Closest match = the classification. No training required.")

# Cross-modal similarity
def cosine(a, b): return (a @ b.T).item()
print(f"\
```

```
Text similarities:")
print(f"  'cat' vs 'dog': {cosine(text_embeds[0], text_embeds[1]):.3f} (similar: both animals)")
print(f"  'cat' vs 'car': {cosine(text_embeds[0], text_embeds[2]):.3f} (different: animal vs vehicle)")
```

📺 **OUTPUT**

```
Text embeddings shape: torch.Size([3, 512])

ZERO-SHOT CLASSIFICATION:
Given an image, which text description matches best?
Just compare the image embedding to each text embedding.
Closest match = the classification. No training required.

Text similarities:
  'cat' vs 'dog': 0.882 (similar: both animals)
  'cat' vs 'car': 0.187 (different: animal vs vehicle)

# Both text AND images live in the same 512-dimensional space.
# This is what powers visual search, zero-shot classification, and multimodal RAG.
```

Multimodal RAG

Retrieve across text, images, tables, audio

> Remember regular RAG? Search text documents, stuff into prompt, generate answer. Multimodal RAG extends this to ALL content types: text, images, charts, tables, and even audio transcripts. Because real documents are not just text -- they have charts, diagrams, photos, and tables that contain crucial information.

6. Cross-Modal Retrieval + Production Patterns

📖 DEEP TEACH

```
THE CHALLENGE:
Imagine a financial report with 3 types of content:
  PAGE 5: text paragraph explaining Q3 results
  PAGE 6: a bar chart showing revenue breakdown
  PAGE 7: a table of regional sales numbers

Regular RAG only reads the text. It MISSES the chart and the table.
Multimodal RAG handles ALL three.

3 PRODUCTION APPROACHES (from simplest to most advanced):

1. CONVERT EVERYTHING TO TEXT (easiest -- start here):
Tables -> convert to markdown format. LLMs understand markdown tables well.
Charts/images -> ask GPT-4o to DESCRIBE them. Store descriptions as text chunks.
Audio -> Whisper transcribes to text.

Then embed ALL the text (original + descriptions) in the same vector database.
Your existing text-only RAG pipeline works unchanged.

This gives 90% of the value with zero infrastructure changes.

2. MULTIMODAL EMBEDDINGS (native cross-modal search):
Use CLIP or Jina-CLIP to embed BOTH text and images in the same vector space.
When user asks about 'revenue chart,' the query embedding is close to the chart embedding even though one is text and the other is
an image.

More sophisticated but requires multimodal embedding models.

3. VISION-AUGMENTED GENERATION (most accurate for complex visuals):
After retrieving relevant pages/images, send them DIRECTLY to GPT-4o or Claude as images.
The VLM 'looks at' the chart and reads data directly from it.
Most accurate for complex charts and diagrams. Most expensive (image tokens).

BEST PRODUCTION PATTERN (combine all three):
1. Convert tables to markdown + describe images with GPT-4o (one-time processing)
2. Embed everything as text in vector DB (simple search)
3. When a query matches an image/chart, ALSO send the original image to the VLM (maximum accuracy)

AUDIO IN RAG:
Podcasts, meetings, customer calls -> Whisper transcribes -> text chunks -> embed -> searchable
Now you can ask: 'What did the CEO say about Q4 projections in the earnings call?'
And the system finds the exact moment in the audio.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like a universal library. A regular library only catalogs books (text). A multimedia library also catalogs paintings
(images), recordings (audio), and data tables. You want the universal one.

1. START WITH APPROACH 1 (convert to text). It gives 90% of the value with zero infrastructure change.
2. Tables -> markdown is surprisingly effective. LLMs understand markdown tables well.
3. Image description: ~$0.01 per image. For 10,000 images = $100 one-time cost.
4. Audio: Whisper at $0.006/minute. A 1-hour meeting = $0.36 to transcribe.
5. Unstructured library: pip install unstructured. Handles PDF/DOCX/HTML parsing automatically.
6. For charts: BOTH describe the chart AND keep the original image. Use the description for search, but send the original image to
GPT-4o for maximum accuracy when answering.
```

WAR STORY: Consulting firm had 10,000 client presentations (PowerPoint files). Partners needed to search for specific charts and findings across all presentations. Before: hire an analyst to manually search (days). After multimodal RAG: GPT-4o described every chart, embedded all text + descriptions. Partners search in natural language. 'Find all presentations showing declining market share in Southeast Asia.' Results in 3 seconds including the actual charts.

📋 POCKET NOTES

3 Approaches (simple to advanced):
  1. Convert to text: tables->markdown, images->GPT-4o description, audio->Whisper
     90% of value. Zero infra changes. START HERE.
  2. Multimodal embeddings: CLIP/Jina-CLIP. Text+images in same space.
  3. Vision-augmented: send actual images to GPT-4o. Most accurate.
Best production pattern: convert+embed as text, but also send original images to VLM.
Audio RAG: Whisper transcribes -> chunk -> embed -> searchable
Costs: image description $0.01/img | Whisper $0.006/min
Unstructured library: Swiss Army knife for document parsing.
Tables as markdown: surprisingly effective.

🎯 INTERVIEW PREP

SAY THIS: 'For multimodal RAG, I use a pragmatic layered approach. Tables become markdown, images get GPT-4o descriptions, audio gets Whisper transcription. Everything becomes text in the same vector space. For complex charts, I also send the original image to a VLM for maximum accuracy.'

Q: How do you make charts searchable?
A: 'Two-pass approach: GPT-4o generates a detailed text description of each chart including all data points. That description is embedded for search. When a user query matches a chart, I send BOTH the text description and the original chart image to the LLM for the most accurate answer.'

Q: How do you handle audio in RAG?
A: 'Whisper transcribes to text with timestamps. Transcription is chunked and embedded like any document. When a user asks about something said in a meeting, the system retrieves the relevant chunk AND can point to the exact timestamp.'

⌨️ CODE

Python

```python
# Multimodal RAG: text + images + tables + audio in one system
from openai import OpenAI
client = OpenAI()

# 1. TABLE -> MARKDOWN (LLMs understand this perfectly)
table = [["Region","Q3 Revenue","Growth"],
         ["North America","$45M","+12%"],
         ["Europe","$32M","+8%"],
         ["Asia Pacific","$28M","+22%"]]

md = "| " + " | ".join(table[0]) + " |\
" + "|---" * len(table[0]) + "|\
"
for row in table[1:]:
    md += "| " + " | ".join(row) + " |\
"
print("TABLE -> MARKDOWN:")
print(md)

# 2. IMAGE -> TEXT DESCRIPTION
print("IMAGE -> TEXT DESCRIPTION:")
desc = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role":"user","content":[
        {"type":"text","text":"Describe this chart in detail for a search system. Include all data points."},
        {"type":"text","text":"[Simulated: bar chart showing NA $45M, EU $32M, APAC $28M Q3 revenue]"}
    ]}]
).choices[0].message.content
print(f"  {desc[:100]}...")

# 3. AUDIO -> TEXT (Whisper)
```

```
print("\
AUDIO -> TEXT:")
print("  transcript = client.audio.transcriptions.create(model='whisper-1', file=audio)")
print("  Cost: $0.006/minute. 1-hour meeting = $0.36")

print("\
ALL content types -> text -> same vector database -> one search interface.")
print("User asks any question. System searches across text, charts, tables, and audio.")
```

OUTPUT

```
TABLE -> MARKDOWN:
| Region | Q3 Revenue | Growth |
|---|---|---|
| North America | $45M | +12% |
| Europe | $32M | +8% |
| Asia Pacific | $28M | +22% |

IMAGE -> TEXT DESCRIPTION:
  Bar chart showing Q3 regional revenue. North America leads at $45M (+12%)...

AUDIO -> TEXT:
  transcript = client.audio.transcriptions.create(model='whisper-1', file=audio)
  Cost: $0.006/minute. 1-hour meeting = $0.36

ALL content types -> text -> same vector database -> one search interface.
```

LIBRARIES

openai	GPT-4o vision, DALL-E, Whisper, TTS	`pip install openai`
diffusers	Stable Diffusion, ControlNet	`pip install diffusers`
transformers	CLIP, LLaVA, Whisper local	`pip install transformers`
whisper	Speech-to-text (local)	`pip install openai-whisper`
Pillow	Image processing	`pip install Pillow`
clip	Multimodal embeddings	`pip install open-clip-torch`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```mermaid
graph TB
    subgraph Input Sources
        Text[Text Documents]
        Images[Images / Charts / Screenshots]
        Audio[Audio / Meetings / Calls]
        Tables[Tables / Spreadsheets]
        PDF[Scanned PDFs]
        Video[Video — emerging]
    end

    subgraph Convert to Text [Step 1: Convert Everything to Text]
        Text -->|Already text| TextChunks[Text Chunks]
        Tables -->|Markdown format| TextChunks
        Images -->|GPT-4o describes<br/>~$0.01/image| TextChunks
        Audio -->|Whisper transcribes<br/>$0.006/min| TextChunks
        PDF -->|OCR: Tesseract /<br/>AWS Textract| TextChunks
        Video -->|Frame extraction +<br/>GPT-4o + Whisper| TextChunks
    end

    TextChunks --> Embed

    subgraph Unified Embedding [Step 2: Embed All as Text]
        Embed[Embedding Model<br/>text-embedding-3-small]
        Embed --> VDB[(Vector Database<br/>All content types together)]
    end

    User([User Query]) --> QueryEmbed[Embed Query]
    QueryEmbed --> Search

    subgraph Search [Step 3: Unified Search]
        VDB --> Dense[Dense Search]
        BM25_Store[(BM25 Index)] --> Sparse[Sparse Search]
        Dense --> Merge[RRF Merge]
        Sparse --> Merge
```

```
    end

    Merge --> Results

    subgraph Results [Results with Source Types]
        R_Text[📝 Text chunk]
        R_Table[📊 Table markdown]
        R_Image[📷 Image description]
        R_Audio[🎙 Audio transcript]
    end

    Results --> Generator[LLM Generator<br/>Answers from mixed sources]
    Generator --> Response([Response with<br/>multi-source citations])

    subgraph Vision Models [Vision-Language Models]
        GPT4o[GPT-4o: best all-rounder<br/>detail:low = 85 tokens $0.00001<br/>detail:high = 1000+ tokens]
        Claude_V[Claude Vision:<br/>dense documents]
        Gemini[Gemini: multi-image<br/>large context]
    end

    subgraph Image Gen [Image Generation]
        DALLE[DALL-E 3: $0.04/img<br/>best prompt understanding]
        SD[Stable Diffusion: ~$0.001/img<br/>your GPU, full control]
        Mid[Midjourney: most aesthetic]
    end

    subgraph Audio Pipeline [Voice Agent Pipeline]
        direction LR
        Hear[Whisper<br/>hear] --> Think[LLM<br/>think]
        Think --> Speak[TTS<br/>speak]
        Speak -.->|Target: < 1 second<br/>with streaming| Done([User hears])
    end

    subgraph Multimodal Embeddings [Alternative: Native Multimodal]
        CLIP[CLIP / SigLIP<br/>Text + images in SAME<br/>vector space]
        CLIP --> VisualSearch[Visual search:<br/>text query finds images]
    end

    style TextChunks fill:#3fb950,color:#fff
    style Embed fill:#1f6feb,color:#fff
    style VDB fill:#8957e5,color:#fff
    style Generator fill:#1f6feb,color:#fff
    style Response fill:#3fb950,color:#fff
    style GPT4o fill:#3fb950,color:#fff
    style DALLE fill:#1f6feb,color:#fff
```

— POCKET NOTES —

1. Image Understanding + Chart Reading + Use Cases

```
VLMs: LLMs that can see. Image -> visual tokens -> processed with text.
GPT-4o: best all-rounder. Photos, charts, docs, screenshots.
Claude Vision: great for dense documents and long screenshots.
Gemini: native multimodal. Multi-image. Large context.
LLaVA: open-source, free, run locally. Privacy-friendly.
Image tokens: 85 (low detail) to 1000+ (high detail) per image.
Use 'detail: low' when possible to save cost.
Use cases: OCR, chart extraction, UI testing, document processing.
Combine with instructor for typed data extraction from images.
Resize images to minimum needed resolution (save tokens).
```

2. Text-to-Image + ControlNet + img2img + Inpainting

```
How it works: start with noise, gradually denoise guided by text description.
Stable Diffusion: open-source, free, your GPU. Full control. diffusers library.
DALL-E 3: OpenAI API. Best prompt following. ~$0.04-0.08/image.
Midjourney: most aesthetic. Discord-based.
4 Techniques:
  Text-to-image: describe -> get image
  img2img: starting image + changes -> modified image
  Inpainting: mark region -> replace just that part
  ControlNet: structural guide (pose/edges) + description -> consistent output
Prompt tips: be descriptive, include style words, use negative prompts
Cost: DALL-E ~$0.04/img | SD on GPU ~$0.001/img
pip install diffusers transformers accelerate
```

3. Speech-to-Text + Text-to-Speech + Voice Agents

```
Speech-to-Text: Whisper. 100+ languages. API $0.006/min. Self-host: free.
Text-to-Speech:
  OpenAI TTS: 6 voices, $15/1M chars. Good balance.
  ElevenLabs: best quality, voice cloning. $5-330/month.
  Coqui TTS: open-source, free, run locally.
Voice Agent pipeline: speak -> Whisper -> LLM -> TTS -> hear
Latency challenge: target < 1 second total
  Stream everything. Groq for fast LLM. Smaller Whisper model.
OpenAI Realtime API: optimized voice pipeline.
Voice cloning: 30 seconds of audio. ALWAYS get consent.
Ethical: deepfakes, impersonation risk. Legal requirements.
```

4. Current Models + Architecture + Practical Reality

```
Sora (OpenAI): most hyped. 60s videos. Limited access. Not production-ready.
Runway Gen-3: most accessible. Text/image to video. Creative professionals.
Kling: competitive quality. Longer clips.
Stable Video Diffusion: open-source. Image-to-video. Experimentation.
How it works: like image gen but with time dimension. Denoise frame by frame.
Reality check:
  Slow (minutes-hours per video)
  Inconsistent quality (physics violations)
  Expensive (GPU-intensive)
  Short clips (5-60 seconds)
Works today: social clips, product animations, concept viz
Not ready: long-form, real-time, precise control
2-3 years from mainstream production readiness.
```

5. CLIP + SigLIP + Cross-Modal Search

```
Multimodal embeddings: text AND images in the SAME number space.
  'photo of a dog' (text) and (actual dog photo) get similar numbers.
CLIP (OpenAI): pioneer. 400M image-text pairs. Two encoders.
SigLIP (Google): improved CLIP. Better retrieval.
Jina-CLIP: production-optimized. Long text (8K). Good for RAG.
Enables: text-to-image search, image-to-text, zero-shot classification.
Zero-shot: classify images into ANY categories without training.
Same vector DB holds both text + image embeddings.
Dimensions: 512-768 typically.
Limitation: CLIP understands WHAT, not spatial WHERE.
```

6. Cross-Modal Retrieval + Production Patterns

```
3 Approaches (simple to advanced):
  1. Convert to text: tables->markdown, images->GPT-4o description, audio->Whisper
     90% of value. Zero infra changes. START HERE.
  2. Multimodal embeddings: CLIP/Jina-CLIP. Text+images in same space.
  3. Vision-augmented: send actual images to GPT-4o. Most accurate.
Best production pattern: convert+embed as text, but also send original images to VLM.
Audio RAG: Whisper transcribes -> chunk -> embed -> searchable
Costs: image description $0.01/img | Whisper $0.006/min
Unstructured library: Swiss Army knife for document parsing.
Tables as markdown: surprisingly effective.
```

— INTERVIEW PREP —

1. Image Understanding + Chart Reading + Use Cases

```
SAY THIS: 'I use GPT-4o for general image understanding and Claude Vision for dense document analysis. For privacy-sensitive use cases,
LLaVA runs locally. The key cost consideration is image tokens -- I use low-detail mode when possible and resize images to minimize
token usage.'

Q: How do you extract structured data from images?
```

A: 'Vision model describes the image, then I pipe the description through instructor with a Pydantic schema for typed extraction. For charts, I ask the model to extract the data points as JSON. For invoices, I define an Invoice schema and the model fills it from the image.'

Q: GPT-4o vs Claude Vision vs Gemini for images?
A: 'GPT-4o: best general quality. Claude: best for long documents with many details. Gemini: best for multi-image comparison and video frames. LLaVA: best for privacy (runs locally). I benchmark on my specific image types before choosing.'

2. Text-to-Image + ControlNet + img2img + Inpainting

SAY THIS: 'For production image generation, I choose based on control needs. DALL-E 3 for quick, high-quality results via API. Stable Diffusion with ControlNet for consistent, controllable generation at scale. The key production technique is ControlNet -- it locks structure while allowing creative variation.'

Q: How do you ensure consistent image style across a product?
A: 'ControlNet with a fixed structural guide (edge detection or pose estimation) plus a consistent style prompt and seed. This gives the same composition and style while allowing content variation. For brand consistency, I fine-tune Stable Diffusion on brand images.'

Q: DALL-E 3 vs Stable Diffusion for production?
A: 'DALL-E 3: zero infrastructure, best prompt understanding, pay per image. Stable Diffusion: full control, unlimited generation, but need GPU infrastructure. Crossover: at ~1000 images/day, self-hosted SD becomes cheaper.'

3. Speech-to-Text + Text-to-Speech + Voice Agents

SAY THIS: 'For voice agents, the pipeline is Whisper for speech-to-text, LLM for reasoning, and TTS for speech output. The critical engineering challenge is latency -- I target under 1 second by streaming each component and using fast inference like Groq.'

Q: How do you build a real-time voice agent?
A: 'Pipeline: Whisper -> LLM -> TTS with streaming at every step. Key optimizations: start TTS before LLM finishes (stream sentence by sentence), use Groq or a fast local model, and smaller Whisper model for real-time. OpenAI Realtime API handles this pipeline natively.'

Q: Voice cloning concerns?
A: 'Always get explicit consent. Many jurisdictions have laws. Use cases: brand voices (consented), audiobook narration, accessibility. Never for impersonation. Implement voice watermarking when possible.'

4. Current Models + Architecture + Practical Reality

SAY THIS: 'Video generation is impressive but not production-ready for most use cases. Runway Gen-3 is the most practical for short creative clips. I am honest about limitations -- temporal consistency, physics violations, and generation time are real constraints. I track this space monthly because it evolves rapidly.'

Q: Should we use video generation in production?
A: 'For specific narrow use cases, yes: social media teasers, product animations, concept visualization, B-roll. For polished long-form content: not yet. The quality is not consistent enough. I recommend a hybrid approach -- AI for ideation and rough cuts, human refinement for final output.'

Q: When will video gen be production-ready?
A: 'For short creative clips: now (Runway). For reliable long-form content: likely 2-3 years. For real-time generation: 3-5 years. The trajectory is similar to image generation, which went from novelty to production-grade in about 3 years (2022-2025).'

5. CLIP + SigLIP + Cross-Modal Search

SAY THIS: 'I use multimodal embeddings to put text and images in the same vector space. This enables cross-modal search, zero-shot image classification, and multimodal RAG where both text chunks and images are searchable from one query.'

Q: How does zero-shot image classification work?
A: 'Embed the image with CLIP. Embed the category names as text. Compare distances. The closest text embedding IS the classification. No training needed. Just provide category names.'

Q: How do you build visual search?
A: 'Embed all product images with CLIP. Store in a vector DB. When user uploads a photo, embed it with the same model and find nearest neighbors. Took 2 days vs 6 months for a custom model.'

6. Cross-Modal Retrieval + Production Patterns

SAY THIS: 'For multimodal RAG, I use a pragmatic layered approach. Tables become markdown, images get GPT-4o descriptions, audio gets Whisper transcription. Everything becomes text in the same vector space. For complex charts, I also send the original image to a VLM for maximum accuracy.'

Q: How do you make charts searchable?
A: 'Two-pass approach: GPT-4o generates a detailed text description of each chart including all data points. That description is embedded for search. When a user query matches a chart, I send BOTH the text description and the original chart image to the LLM for the most accurate answer.'

Q: How do you handle audio in RAG?
A: 'Whisper transcribes to text with timestamps. Transcription is chunked and embedded like any document. When a user asks about something said in a meeting, the system retrieves the relevant chunk AND can point to the exact timestamp.'